

Categorize more, without trading away precision

An AI transaction categorization engine behind client ESG reporting, gated on confidence and released only when it passed an evaluation.

Akshay Dhar · akshaykumardhar14@gmail.com · akshaykumardhar.github.io

MY ROLE

Data Consultant. Architected and led the build with a small team, and owned the engine from design through production.

WHEN

Apr - Sep 2026

STACK

Python, SQL/PostgreSQL, SBERT (MiniLM, E5, BGE), YAML configs, Metabase

39% → 13%

uncategorized transactions

+5 pts

category accuracy from an embedding bake-off

21 → 36

categories, zero golden-set regression

Days → hours

to onboard a new banking partner

The problem

At Catalysk (AI sustainability analytics, Bengaluru), bank-statement transactions feed client ESG reporting, and **39% of transactions landed uncategorized**. Every uncategorized row is manual analyst work. Every wrongly categorized row is an error in a client's report.

So the goal was never "categorize more". It was **categorize more without trading away precision**, and prove it before each release.

How the engine works

Ingest

CSV, Excel, JSON and REST through one contract

Rules

keyword and rule matching for the clear cases

Merchant extraction

who was paid

SBERT fallback

semantic match for the rest

Confidence gate

~84%: below it, the model abstains

Auto-categorize

or route to analyst review

An evaluation harness, a 388-row UAT golden set plus a 1,400-row benchmark across 7 bank statements, gates every model and taxonomy change. An 18-table PostgreSQL observability layer with live Metabase dashboards replaced manual Excel review.

Six product calls that made it work

Define "good" before building

Built the release bar first: a 388-row UAT golden set and a 1,400-row benchmark across 7 bank statements. No model or taxonomy change shipped without passing it.

21 → 36 categories (10K → 17K training examples), zero golden-set regression

Precision over vanity coverage

Gated auto-categorization at ~84% model confidence. Below the gate, the model abstains and the row goes to an analyst instead of into a client report.

Uncertain rows get human review, not a guess

Pick models on evidence, not reputation

Ran an embedding bake-off (MiniLM, E5, BGE) against the golden set, and rejected a slower model that only looked better on paper.

+5 points category accuracy

Go after the misses

Redesigned the categorization logic for UPI (VPA) and person-to-person (P2P) payments.

Uncategorized 39% → 13%

Make it fast enough to use

Removed per-row regex recompilation and redundant set rebuilds, and added embedding caching.

Hours → minutes per 1,000 rows, zero change to output

Make it self-serve for non-engineers

Config-driven YAML rules analysts edit without engineering, plus live Metabase dashboards replacing manual Excel review.

Partner onboarding from days to hours

Guarding against silent failure. When a catch-all bucket swelled to ~30% of volume, I traced it to a silent regression that had disabled the SBERT model and 4 downstream stages without raising an error, and recovered it. I also shipped 90+ automated data quality checks across 5 financial schemas, catching silent bad data before it reaches the model.

What I took from it

1. **Write the eval before the feature.** The golden set and benchmark were the spec and the release bar.

2. **Measure precision on what the AI does alone.** Automation without precision just moves the work downstream.
3. **Let the model abstain.** The gate keeps analysts on the rows that need judgment.
4. **Watch for silent failure.** A model that quietly switches off looks like a data problem until you trace it.

Figures are from production work at Catalysk (Apr - Sep 2026). No client data or code is shown.